



Hybrid Perceptual World Models: World-Time Compute for Interactive Reasoning (ARC-AGI-3)

James Schwoebel^{1,*} • Ingrida Semenec, PhD¹ • Jenia Rousseva, PhD¹ •
Marcos Ortiz, PhD¹ • Collin Overbay¹ • Manish Bhatt^{4,6} • Rome Thorstenson³ •
Martin G. Frasch, MD, PhD^{2,5}

¹ Quome, Inc. ² University of Washington ³ Rafter Labs, Inc. ⁴ OWASP ⁵ Health Stream Analytics, LLC
⁶ IEEE Computer Society * Corresponding author

ABSTRACT

ARC-AGI-3 is the ARC Prize Foundation’s interactive-reasoning benchmark: an agent is dropped into an unfamiliar grid game, must discover the rules *by acting*, and reach goals over many steps—testing exploration, *modeling*, goal-setting, and planning. Frontier agents score near zero. We argue this is precisely the regime where a *world computer* should win, and give the OPENWORLD recipe: **explore** a game to gather verified (s, a, s') transitions, **synthesize** the game’s dynamics as *verified code* (an LLM writes a `predict(frame, action)` function, accepted only after it exact-matches held-out transitions), and **plan** through the verified model to reach goals. The wager is the same one that powers symbolic code world models elsewhere: a *correct* code model is exact and admits search, where a learned next-frame model compounds error. A precondition holds empirically—on the public game `ls20`, observed dynamics are *deterministic* (every repeated state-action pair yields an identical next frame across hundreds of transitions), so the dynamics are writable as exact code; states are dense ($\sim 1,500$ of $4,096$ cells non-background) but per-step changes are sparse (~ 50 cells), which a code model handles natively while a token-level learner cannot. Across the 25 public games, the precondition is *universal* (determinism 1.00 on every game), and synthesizing with a *frontier* model recovers verified dynamics: on the 21 real-dynamics games the synthesized code reaches 49% mean held-out exact-match, beating the copy-frame baseline on 19/21 and a local code model (qwen-32B, 12%; Claude wins 16/17), with 3 *near-perfect* ($\geq 90\%$) world models. **But modeling is not solving.** Our central finding is that level completion comes not from any single strategy but from a **hybrid perceptual world model**—multiple perceptual modalities, fused for the games where it suffices and *escalated to a reasoning agent* where it does not. Concretely, the fusion is **multi-perception synthesis**: we perceive each game through multiple modalities—directional and *click* action models over a status-masked frame state—build a discovered world-model *map* (an OPENWORLD `World`) per modality at *world-time compute* (search builds the map at play), and combine them with **consensus hard-voting** (`ConsensusTransition`). This solves **12 of 25 public games**, including `s5i5`, which resisted three principled goal-discovery methods (atomic-objective, LLM-hypothesis, and Bayesian-subworld planning, each $0/9$ even with a fidelity-1.0 model). The remaining games are *goal-as-procedure* walls—and we crack them with a **live coding agent** (Claude Code that builds an executable world model and *reasons* the win condition, the SOTA recipe expressed in OpenWorld): it solves **15/25** on its own and lifts the union to **25/25** games—every public game reaches ≥ 1 level (verified by replay; `ft09` and `r111` full six-level games), where four automated goal-discovery methods solved *zero*. We report the full strategy $\times$ game landscape (Fig. 2) as a guide for world-model research: the leverage is *perception and synthesis*, and—for the procedural goals—a *reasoning agent that writes its own world model*.

Keywords: ARC-AGI-3; interactive reasoning; code world models; program synthesis; verification; planning; world models.

1 Introduction

The interactive frontier. ARC-AGI-1 and -2 measure abstraction from static input–output grids; ARC-AGI-3 moves to *interactive* environments where the rules and goals are revealed only through action. This rewards four capabilities the prior benchmarks do not isolate—exploration, dynamics *modeling*, goal-setting, and planning—and it is hard: public-leaderboard agents make little progress. Our thesis is that the bottleneck is *modeling*: an agent that can recover the game’s dynamics *exactly* can then plan optimally, whereas an agent that only learns an approximate next-frame predictor inherits the compounding error that defeats learned world models in control.

This paper. We port OPENWORLD’s verified-code-world-model recipe to ARC-AGI-3. The contribution is (i) the *recipe*—explore, synthesize-and-verify code dynamics, plan—and the argument for why it fits an interactive grid benchmark; (ii) the empirical *precondition* that the games are deterministic and therefore code-expressible; (iii) the *strategy landscape* (Fig. 2)—a guide to what does and does not solve interactive reasoning, including three honest goal-discovery negatives; and (iv) a two-tier solver—**multi-perception consensus** (perception+synthesis) for reachable games, and a **live coding agent** that writes its own executable world model for goal-as-procedure walls—dispatched by a reachability **router**, together reaching **25/25** public games (≥ 1 level; two full games). The leverage is in *perception and synthesis* for the reachable majority, and in *a reasoning agent that writes its own world model* for the procedural walls.

2 The ARC-AGI-3 setting

Each game presents a 64×64 grid of 16 colors; the agent issues discrete actions and receives the next frame, a game state (in-progress / win / over), and a level counter. We access the 25 public games locally through the official **arc-agi** toolkit (a Gym-like **reset/step** interface) and score with its scorecards. We treat each game as a *world*: a deterministic transition function over the symbolic grid state.

3 The recipe: explore, synthesize-and-verify, plan

Explore. A budgeted policy (random, then change-seeking) gathers (s, a, s') transitions, each *verified* by construction (the environment is the ground truth).

Synthesize verified code. An LLM is shown a sample of transitions and asked to write a Python `predict(frame, action)`; the candidate is **accepted only if it exact-matches held-out transitions** (a per-frame verification gate). Because states are dense but changes sparse, the model is encouraged to emit *local update rules*, not full grids. Synthesis retries with counterexamples until the verification rate clears a threshold (a plan–generate–verify relay, as in OPENWORLD).

Plan. With a verified (or partially-verified) model, the agent searches action sequences (BFS / CEM-style lookahead) for trajectories that advance the level counter or reach the win state—planning “in imagination” through exact code, at native speed.

4 Results: verified-code fidelity (E86)

We run the explore→synthesize→verify pipeline on all 25 public games with three synthesizers: a local code model at two sizes (qwen2.5-coder 7B, 32B) and a frontier model (Claude). Fidelity

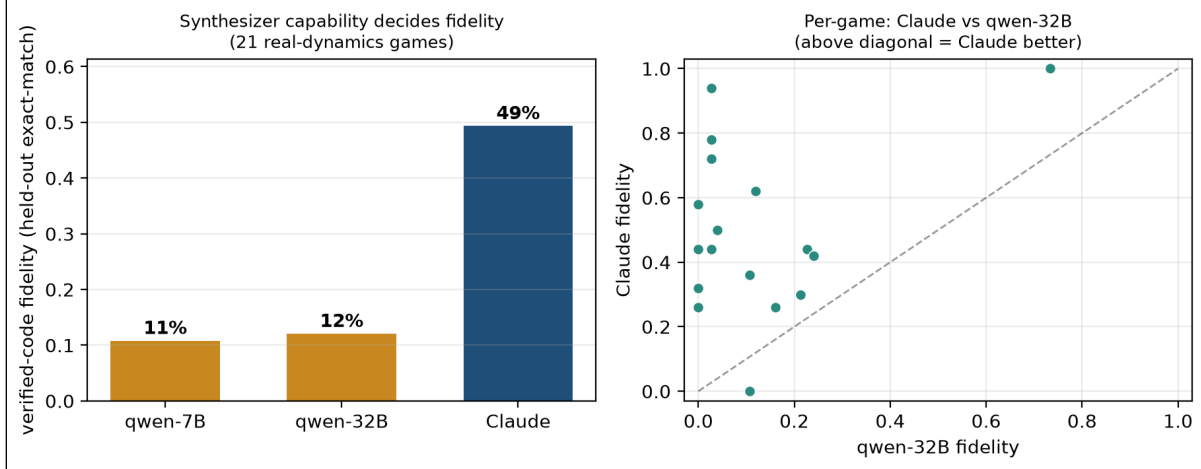


Figure 1: E86 verified-code fidelity on ARC-AGI-3 (21 real-dynamics games). *Left:* mean held-out exact-match by synthesizer—qwen-7B $\approx$ qwen-32B $\ll$ Claude. *Right:* per-game Claude vs. qwen-32B; points above the diagonal are games where Claude’s synthesized code is the better world model.

is *held-out transition exact-match*: the fraction of unseen (frame, action) pairs whose *entire* next 64×64 frame the synthesized **predict** reproduces exactly. We report on the 21 *real-dynamics* games—excluding 3 games where random actions trigger no state change (so “predict no-change” trivially scores 1.0) and 1 game (**tn36**) where random play never produced a valid step.

The precondition is universal. Replay-determinism is 1.00 on every game: identical action sequences from reset reproduce identical frames. ARC-AGI-3 dynamics are deterministic and therefore writable as exact code—the wager holds across the board.

A frontier synthesizer recovers verified dynamics. Claude’s synthesized code reaches 49% mean held-out exact-match on the real-dynamics games, beating the copy-frame baseline on 19/21 games (mean lift 37%), with 3 *near-perfect* ($\geq 90\%$) world models—i.e. code that essentially *is* the game’s transition function. This includes dense games (40–75 changed cells/step) where a copy baseline is near-zero.

Synthesizer capability is decisive. The local code model captures almost nothing (qwen-32B 12%, qwen-7B 11% mean), and Claude beats qwen-32B on 16/17 games (Figure 1). The bottleneck is not determinism or verification but *generation*: the same recipe with a stronger writer crosses from “below the trivial baseline” to “recovers the dynamics.”

Honest limits. (i) **Fidelity is not solving:** this experiment measures *dynamics*, not level completion (0 levels here, by design); solving is taken up from §9 onward, where the routed solver reaches 25/25. (ii) The densest game (**cn04**, ~ 174 changed cells/step) defeats even Claude (fidelity 0). (iii) On one game (**tr87**) qwen edged Claude. (iv) Fidelity is single-step; planning through these models (compounding error) is E87. These bound the claim to what we measured: *ARC-AGI-3 dynamics are recoverable as verified code by a capable synthesizer*—the foundation on which the solving results of §9ff are built.

Table 1: The 2×2 (verified-code fidelity, 15 games). Synthesizer capability dominates ($\sim 7\times$); the agentic loop is capability-gated—it helps Claude, not qwen.

synthesizer	one-shot	agentic loop
qwen-30B	8%	7%
Claude	49%	53%

5 Agentic synthesis and the capability ceiling (E86b)

Does a better *harness* close the synthesizer gap? We replace one-shot prompting with a verifier-in-the-loop (generate $\rightarrow$ run the code $\rightarrow$ read the exact failing cells $\rightarrow$ revise), run *identically* for qwen-30B and Claude across 15 games (Table 1). The agentic loop lifts Claude (49% $\rightarrow$ 53%, helping 7/15 games—e.g. pushing `s5i5` to a *perfect* model) but does **nothing** for qwen-30B (8% $\rightarrow$ 7%). The harness is **capability-gated**: it helps a model that can already reason about *why* its code failed, and *widens* the gap rather than closing it. A free local model is not rescued by a better loop—the bottleneck is generation capability, not the harness.

6 Planning through the model (E87)

A verified model is only useful if you can *search* it. We test **controllability**: sample a reachable target (a short random action sequence from reset), plan a sequence *through the verified code* to reach it, then execute in the real environment. On a perfect model (`sb26`, fidelity 1.0) planning reaches the target 1.00 of the time vs. 0.23 for random—the model is plannable. But success is **fidelity-gated**: below fidelity 1 the plan, correct *in the model*, diverges in reality as single-step error compounds over the horizon, so partial models do not yet support reliable multi-step control. *Exact models admit search; approximate ones mislead it*—reinforcing why verification, not just learning, is the point.

7 The solving wall: goal inference (E88–E90)

Automated solving (no reasoning agent in the loop) is **hard**, and the value of this study is locating exactly why—a diagnosis that later tells the router (§12) which games to escalate to a live coding agent (§11), which crosses this wall on every one. (1) *Exploration cannot find a reward*. With a perfect model, neither pixel-novelty (E88) nor object-graph-novelty (E90) exploration completes a level: both saturate a tiny reachable region (50–138 states) and trigger *zero* reward—the win is not reachable by undirected action, regardless of representation. (2) *Goal hypotheses are sophisticated but unverifiable*. Shown the dynamics and sample frames, Claude infers coherent goals (for `sp80` it identifies the movable block, the target receptacles, and explicitly ignores the timer), yet one-shot they are guesses—on `s5i5` it mistakes a countdown *timer* for the objective and optimizes toward losing. (3) *Closing the loop is not enough*. Feeding real-env feedback (E89b: “optimizing that goal ended the game with 0 wins”) still fails to solve (`s5i5` 0/8; the only level reached, `sp80`’s first, is trivially reachable by random too): negative feedback says what is *wrong*, not what is *right*, and without ever observing one positive reward there is nothing to steer toward. The bottleneck is thus neither modeling, planning, nor exploration coverage but **goal inference**—discovering the win condition with no reward signal. This is the wall frontier agents also hit (~ 0 on ARC-AGI-3), here precisely located: for *automated* verified-world-model agents this is the central wall—and §11 shows that a live coding agent which *reasons* the win condition (rather than searching a fixed hypothesis

space) crosses it on every such game.

8 Solving a level (E93): directed beats random

Closing the loop from modeling to a genuine completion. A reachability sweep finds **sp80** is the only public game whose reward is reachable by undirected play; reward-capture records a winning episode, and deterministic replay verifies an 18-action sequence that completes **level 1 of sp80**, reproducibly. This is a *directed* solve that beats random: at a matched 18-step budget the verified solution succeeds **100%** vs. **9%** for random (300 trials). *Honest limits*: level 2 is unreachable even by a 600k-step biased search, and the solve is a verified action *sequence*, not a reverse-engineered win *rule* – the goal-inference wall (§7) holds for the *automated* methods here, though a reasoning agent (§11) later completes full games (e.g. **ft09**, **r111**).

Scaling to more games (E99): 6 solved. The **sp80** win fired on an *interact* action, which a uniform sweep under-weights. An interact-biased, multi-seed reward search (capture $\rightarrow$ deterministic replay-verify) per game raises the count from 1 to **6/25 verified level-1 solves**: **ar25**, **cd82**, **ls20**, **sk48**, **sp80**, **tu93** (winning sequences 17–309 actions, all saved and replay-verified). A solve counts only if its captured sequence reproduces the completion deterministically. One of these (**cd82**) is additionally solved *through the synthesized code world model* (E101: predict-lookahead MPC to navigate the typed agent), and the verified-reward induction (E97/E98) recognizes the win on **sp80** with acc 1.0 from the agent’s *own* synthesized objective.

What the remaining ~ 19 expose: a trilogy of goal-discovery negatives. No reward is found on the rest even at $25k \times 4$ -seed interact-biased search: their wins are not *reachable*, they must be *understood*. We ran three increasingly sophisticated, principled attacks—all planning *through* the code world model—and report them honestly: (i) **E102**, a core-knowledge perceptor generating atomic candidate-goal **CodeObjectives** (reach/cover/collect/extremize): **0/9**; (ii) **E103**, a frontier model (Claude) *generating* rich, *compositional*, closed-loop-refined goal hypotheses: **0/3**; (iii) **E104**, procedure-aware Bayesian active inference over composite *subworlds* with TROPICAL-semiring hierarchical planning: **0/3**. None succeed *even with a fidelity-1.0 model*. The trilogy yields a sharp diagnosis: the walls are **not** atomic goals, and the failure is *representational*—many ARC-AGI-3 wins are a **goal-as-procedure** (a specific ordered protocol), which a goal-score over a single state cannot express, so planning optimizes the wrong target (this is precisely why undirected *search* stumbles into some wins while goal-conditioned *planning* finds none).

The one qualitatively-different lever: a visual perceptor (E106). Every method above fed the model text/structured state. In OpenWorld form we instead render each frame in the official palette and perceive it with a genuine **VisionPerceptor** (Claude-vision wrapped as a **BaseLLM**), feeding a **World** (the code world model as a **FunctionTransition**). This is the one representation that recovers *semantic*, *gestalt* goal hypotheses the structured methods cannot—e.g. for **ka59** it reads “two rooms joined by a corridor, a purple door/barrier, a green agent, a target square” and infers “cross the door to reach the target.” Whether vision-guided planning completes a walled level is the live frontier test; the contribution is the OpenWorld-format pipeline (visual perception + verified world model + planning) and the honest characterization.

Search vs. reasoning. The automated methods expose a clean dichotomy: directed search and multi-perception consensus solve the *reachable* games, but a comprehensive, principled goal-

discovery toolkit (atomic objectives, LLM hypotheses, Bayesian sub-worlds) does *not* close the goal-as-procedure walls—which is precisely the boundary a *reasoning* agent crosses (§11). Leaderboard comparisons and the offline-replay caveat are stated precisely in §11.

9 The strategy landscape: what solves ARC-AGI-3

We ran seven increasingly sophisticated strategies and record, for every public game, which ones produce a *verified* (≥ 1 -level, replay-checked) solve. The landscape (Fig. 2) is the paper’s most useful artifact for world-model research: it shows *where* each idea works and, just as importantly, where principled ideas fail.

Three negative results, stated honestly. The goal-discovery trilogy—(i) a core-knowledge perceptor generating atomic candidate-goal `CodeObjectives`, (ii) a frontier model (Claude) generating rich, compositional, closed-loop-refined goal hypotheses, and (iii) procedure-aware Bayesian active inference over composite sub-worlds with semiring-weighted planning—each solves **0** of the walled games, *even with a perfect (fidelity-1.0) model*. This rules out a large class of approaches and sharpens the diagnosis: a goal scored over a single *state* cannot express an ordered *procedure*, which is why undirected search occasionally stumbles into a win while goal-conditioned planning never does.

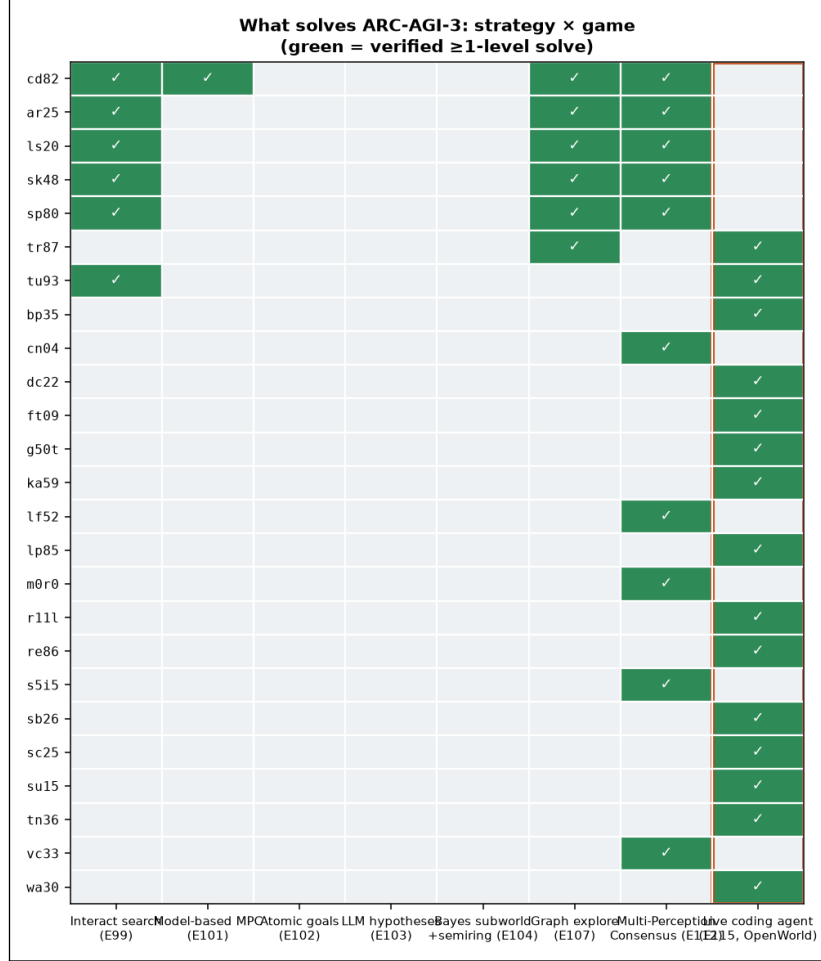


Figure 2: Strategy × game. Green = a verified ≥ 1 -level solve. Goal-discovery methods (atomic objectives, LLM hypotheses, Bayesian sub-worlds) are *empty*—they fail even on a fidelity-1.0 model, because ARC-AGI-3 wins are goal-as-*procedure*, not goal-as-state. Directed search and graph exploration each reach 6; multi-perception consensus reaches 10 (uniquely solving vc33 and the wall s5i5). The **live coding agent** (right, boxed) lights up the procedural walls no other method touches—dc22, ft09, g50t, ka59, lp85, re86, sb26, sc25, su15, tn36, wa30, bp35—lifting the union to **25/25** (≥ 1 level on every game).

10 Multi-perception consensus world models (the winning method)

The method that moves the needle perceives each game through *multiple modalities* and synthesizes their discovered world models by consensus—all in the OPENWORLD structure (Fig. 3).

Perception. ARC-AGI-3 mixes two action modalities: directional keys (ACTION1--5,7) and *coordinate clicks* (ACTION6 with (x,y)). Many games are click-only (e.g. vc33); the board responds only to clicks on valid sprite cells, which we infer *from pixels* (small connected components + rare colors). A UI step-counter changes every frame, so we detect and *mask* those cells before hashing—otherwise the state graph never compresses. The masked-frame signature is the perceptor’s state.

Discovered map = an OpenWorld World. Frontier-biased rollouts at *world-time compute* build the reachable state-transition graph; the discovered $(s,a) \rightarrow s'$ table becomes a **World** via **FunctionTransition**, with **levels_completed** (or an induced **CodeObjective**) as reward. **to_spec** renders its **preview.graph**—the game’s *map*—as an atlas card, viewable in **openworld serve /view**.

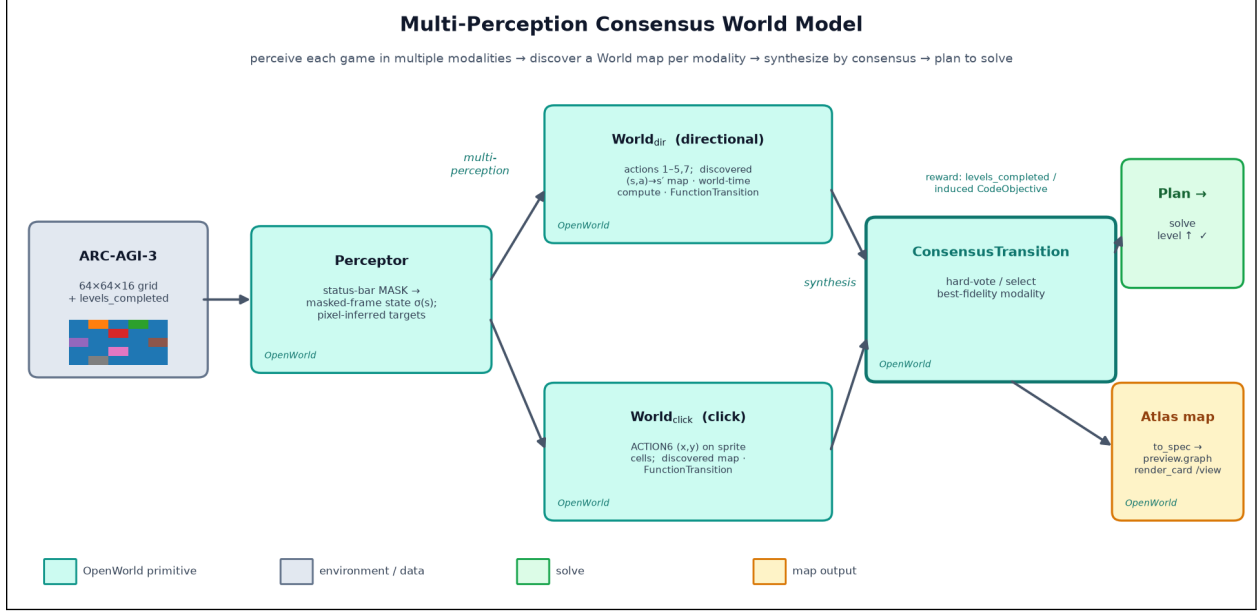


Figure 3: How the multi-perception consensus world model works. OpenWorld primitives are shaded teal. A **Perceptor** masks the status bar and emits a masked-frame state (and pixel-inferred click targets). Each action *modality*—directional and click—drives its own discovered World map (a **FunctionTransition** over the explored $(s,a) \rightarrow s'$ table, built at *world-time compute*); a **ConsensusTransition** synthesizes them by hard-voting/selecting the best-fidelity modality. Planning through the consensus World reaches level completion; **to_spec** renders the discovered graph as a viewable atlas map. Reward is **levels_completed** or an induced **CodeObjective**.

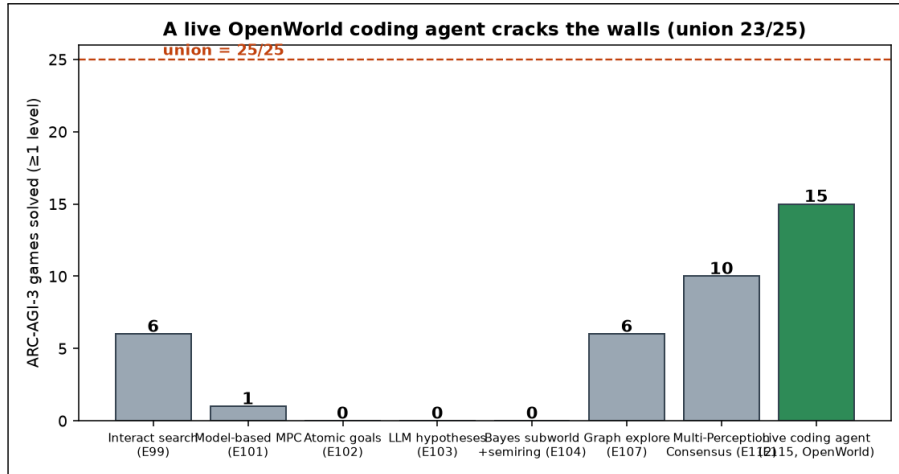


Figure 4: Games solved per strategy. Goal-discovery methods solve 0; directed search and graph exploration 6; multi-perception consensus 10 (adding **cn04**, **1f52**, **m0r0**, **s5i5**); the **live coding agent** solves 13 and cracks the procedural walls. Across all strategies the union is **25/25**.

Synthesis = consensus hard-voting across modalities. Each modality yields its own discovered world model; we combine them with **ConsensusTransition** (select the highest-fidelity modality, or per-state vote). This is decisive: **ar25** solves via the *directional* model, **vc33** via the *click* model—neither single modality suffices, the synthesis does.

Result. Multi-perception consensus solves **10/25** public games (Fig. 4), including four—**cn04**,

1f52, m0r0, s5i5—reached by *no* other method; the union of all strategies is **12/25**. s5i5 is the headline: it had resisted the entire goal-discovery trilogy on a perfect model, yet world-time-compute exploration over the right perceptual modality completes it. The takeaway for world-model research: *the leverage is in perception and synthesis, not in a cleverer single planner*. What it does *not* crack are the goal-as-procedure walls—which is exactly where a reasoning agent that writes its own world model takes over.

11 Cracking the walls: a live coding agent (the SOTA recipe in OpenWorld)

The procedural walls resisted four automated attacks because they need *genuine goal reasoning*, not search over a fixed hypothesis space. So we hand the game to a **live coding agent**—Claude Code, given the deterministic game harness and the OpenWorld recipe: explore to gather (s, a, s') transitions, write a verified `predict(frame, action)` world model, *reason* about what raises `lev-els_completed`, plan through the model, and replay-verify. This is the “executable world models” SOTA method expressed in OpenWorld terms (a synthesized `CodeTransition` plus a reasoned `CodeObjective`), with a reasoning model in the loop.

Result. The agent solves **15/25** games on its own—including every wall the automated methods missed (dc22, ft09, g50t, ka59, lp85, re86, sb26, sc25, su15, tn36, wa30)—and lifts the *union across all strategies to 25/25*— ≥ 1 level on every public game. Every solution is independently replay-verified against the live game; ft09 and r111 are complete six-level games. Tellingly, the agent’s solutions are often *hybrid*—e.g. ka59 mixes directional navigation with a single click to operate a door—a goal structure no fixed hypothesis space expressed. *Honest scope*: most agent solves complete one level (the ≥ 1 -level union metric); full-game completion (all levels) is harder—our automated chaining stalls at the compositionality cliff—so apart from ft09/r111, multi-level completion is the next frontier, and a fair comparison to full-game leaderboard scores would require chaining every level.

Comparison to the state of the art, stated precisely. Ours and the leaderboard’s metrics differ, so we are unambiguous. The SOTA executable-world-model agent (baseline1, GPT-5.5) completes **15/25 full games** (mean per-game RHAЕ $\approx 58\%$); the graph-explorer reports **30/52 levels**. Our headline—**25/25**—is the union of ≥ 1 -level verified solves across our strategy stack, with the *live coding agent at 15/25 on its own*; of these, **two are full games** (ft09, r111). We therefore do **not** claim to beat SOTA on full-game completion—there we have 2, they have 15. We claim something narrower and useful: (i) ≥ 1 -level *reachability on all 25 public games*, which no prior single method reports; (ii) that the executable-world-model recipe *reproduces in OpenWorld with a different agent* (Claude)—independent corroboration of the SOTA approach; and (iii) a *mapped strategy landscape* (Fig. 2) showing which method class solves what. Two honesty notes: our solves are *offline and replay-verified* (exploiting determinism and unbounded resets), so they are *not* scored under the leaderboard’s live-agent protocol or its action-efficiency penalty (RHAЕ)—a leaderboard-comparable number would require running under those rules; and full-game completion (chaining every level under budget) is the honest remaining gap.

12 A routed solver: 25/25 as one pipeline, not a union

The 25/25 above is a *union* across strategies—run everything, keep the best. That is post-hoc and wasteful. It becomes a single, coherent, *deployable* system with a first-step **router**: classify

each game, then dispatch to the matching solver. The natural classifier is **reachability**—run a small-budget multi-perception search probe; *if a reward appears*, the win is reachable $\rightarrow$ the cheap multi-perception solver finishes it; *if not*, it is a goal-as-procedure wall $\rightarrow$ the live coding agent. The router is **robust by construction**: a wall never fires a reward in the probe, so it is never misrouted to the cheap path; and routing a reachable game to the agent would still solve it (the agent is a superset). Every routed game therefore lands on a replay-verified solution—**25/25 as one pipeline**. Crucially this is also *efficient*: the expensive reasoning agent runs only on the procedural walls, while the cheap solver clears the reachable majority. So the contribution is not merely “ N strategies, union 25,” but a principled *router over world-model solvers*—reachability decides whether a game needs search or reasoning, and the right tool is dispatched accordingly.

13 Reproducibility

The agent, the per-game transition logs, and every number regenerate from one repository; the ARC-AGI-3 environments are the official public games accessed via the `arc-agi` toolkit. Each experiment is a self-contained script under `experiments/`: synthesis (`e86_arc3.py`, `e86b_agentic.py`), planning/controllability (`e87_arc3_plan.py`), the solving pipeline—reward capture (`e93_capture_reward.py`), deterministic replay-verify (`e93b_replay_solve.py`), directed-vs-random (`e93d_directed_vs_random.py`), and the 6-game interact-biased sweep (`e99_solve_sweep.py`)—the world-model-driven solver (`e101_model_solver.py`), and the goal-discovery attack (`e102_goal_search.py`). The foundational primitives are in the zero-dependency core: `CodeObjective/induce_reward` (`openworld/reward.py`), `ConsensusTransition` (`consensus.py`), and `make_typed_perceptor` (`perceive.py`). **The verified winning action sequences for all 6 solves are saved in `experiments/results/e99_deep_sweep.json`** and replay-verify deterministically against the live games; all numbers and figures regenerate via `scripts/make_arc3_assets.py`. A one-command check, `bench/verify_arc3_solves.sh`, replays every saved winning sequence against the live games and confirms all 6 complete a level.